Dynamic Branch Prediction Unit

Project requirements and implementation guide

This guide explains what our Modern Computer Architecture project asks us to build, how our Verilog implementation works, and how we demonstrate that it is correct. It is written for team members who need to understand and present the project without reading every source file first.

The project in one paragraph

A processor must choose which instruction to fetch next before every branch outcome is known. Our hardware predicts that choice, learns from resolved branches, and requests a flush and redirect when the saved prediction is wrong. We compare a simple baseline with two predictors that learn from past outcomes.

Strategy	How it chooses a direction
Always not taken	Always continue to the next sequential instruction.
1-bit predictor	Use the last outcome stored for the indexed branch.
2-bit predictor	Use a saturating counter that tolerates an isolated outcome change.

What we chose to build

We implemented a standalone, synthesizable prediction unit with a branch-resolution test harness. Project 5 lists predictor RTL and evaluation deliverables, so a complete CPU is outside our chosen scope. The harness supplies known branch addresses and outcomes, saves the original prediction, and resolves each branch two cycles later.

Where to start

Run make demo for a short explanation in the terminal. Open reports/results.html for the visual comparison across all workloads. Use the remaining pages to understand the requirements, hardware, verification, and demonstration.

The measurements in this guide come from the project CSV and JSON evidence. CPI and FPGA frequency values are analytical or tool estimates; no physical board measurement is claimed.

Project requirements

The supplied brief defines Project 5 and a set of requirements common to all projects. The matrix below maps those requirements to our implementation. Module suggestions and example instructions in the brief guide the design without fixing its exact size or interface.

Requirement from the brief	How our project meets it
Architecture diagram and explanation	The system diagram shows direction prediction, target storage, address selection, and recovery.
State transition description	A four-state diagram explains the 2-bit counter, including both saturation boundaries.
Modular synthesizable Verilog RTL	Separate modules implement the history table, target buffer, prediction selection, and recovery interface.
Dedicated testbench and branch traces	Directed tests cover corner cases; ten deterministic workloads supply branch histories.
Functional simulation and waveforms	Icarus runs the RTL; saved VCD files and a waveform preview expose signal timing.
Accuracy and strategy comparison	All three strategies run identical traces and produce direction and next-address accuracy tables.
Quantitative performance metric	Recovery counts and analytical CPI estimate the effect of wrong next-address predictions.
Synthesis resources and maximum frequency	Yosys and nextpnr produce device-specific utilization and routed timing estimates.
Final report and live demonstration	The technical report records evidence; the demo guide supplies a presentation sequence.

Design choices rather than course mandates

Our defaults are 32-bit addresses, 4-byte instructions, 64 history entries, 16 target entries, a two-cycle resolution delay, and a two-cycle recovery penalty in the CPI model. These are our practical implementation choices. The brief does not mandate these numbers.

We use BEQ and BNE examples, as suggested in the brief. Each outcome is derived from operand equality or inequality. FPGA implementation is suggested where hardware is available; our delivery uses simulation and synthesis without a physical board.

Source: Project_list_MCA.pdf, Project 5 on pages 3 and 4, and Common Project Requirements and tool flow on page 7, as inspected when the project was scoped.

How branch prediction works

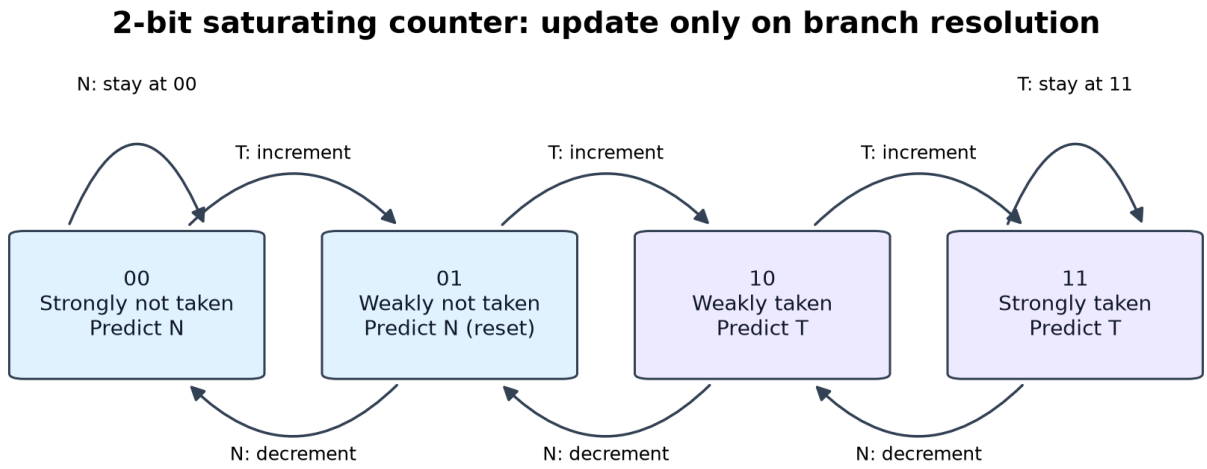
The program counter, or PC, is the address of the current instruction. For our fixed 4-byte instruction format, ordinary sequential execution continues at PC + 4. A taken branch chooses a target address instead. BEQ is taken when two operands are equal; BNE is taken when they differ.

Direction and destination are separate questions

The history table predicts taken or not taken. The branch target buffer, or BTB, stores a destination. We use the target only when direction is taken and the BTB has a valid matching entry. Otherwise we predict PC + 4. This is why high direction accuracy alone does not guarantee a correct next address.

Why the second bit helps a repeated loop

A 1-bit entry changes to not taken at loop exit, so it often misses the next invocation's first taken branch. A 2-bit counter can remain weakly taken after one exit. Its prediction is the counter's most significant bit. Taken increments it; not taken decrements it; the value never wraps past either end.

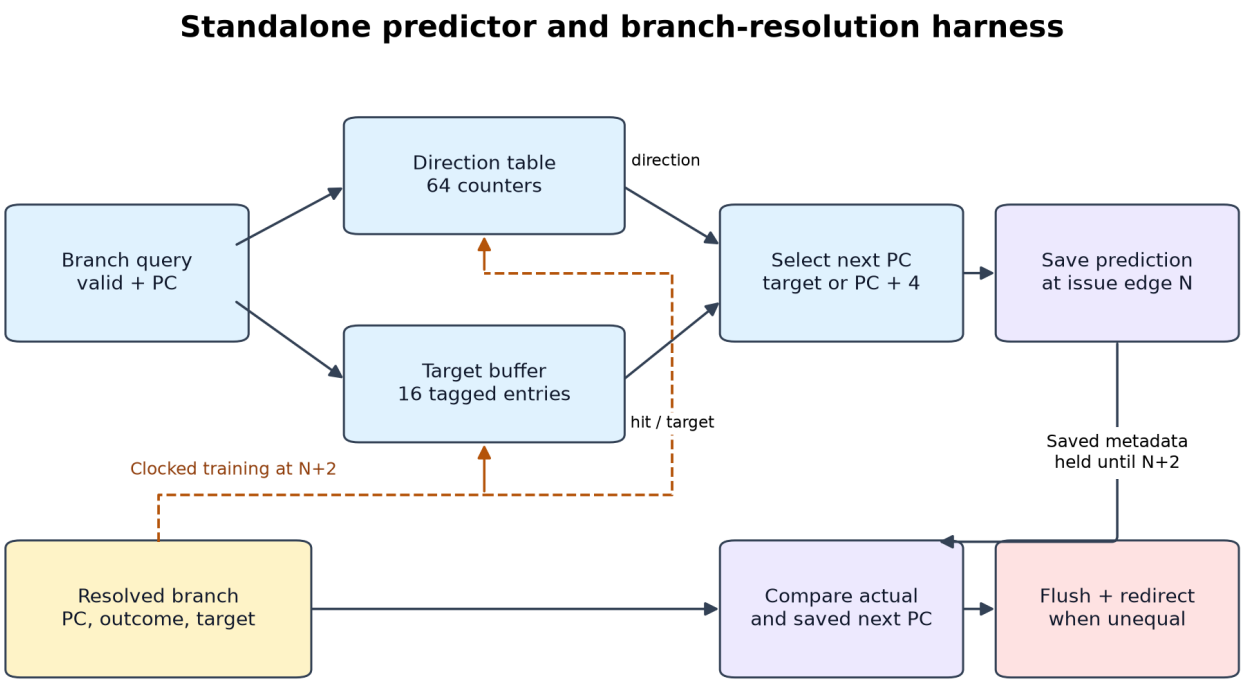


The 2-bit counter resets to 01 and updates only on a valid resolved branch.

Counter state	Prediction	If taken	If not taken
00 Strongly not taken	Not taken	01	00
01 Weakly not taken	Not taken	10	00
10 Weakly taken	Taken	11	01
11 Strongly taken	Taken	11	10

An outcome can flip a weak state immediately; the predictor does not always need two misses to change direction. Its resistance to a single contrary outcome applies when it starts in a strong state. Alternating outcomes can defeat both dynamic predictors with our chosen reset states.

Our hardware design



Prediction is saved at issue edge N and checked against the resolved next address at N+2.

The modules and their responsibilities

direction_predictor.v stores the 1-bit or 2-bit history, or implements the fixed baseline. The default index is PC bits 7 through 2. Entries have no tag, so addresses 256 bytes apart share a history entry and can interfere.

branch_target_buffer.v stores 16 valid bits, address tags, and target addresses. PC bits 5 through 2 select an entry. A taken resolution replaces that slot; a not-taken resolution leaves its target intact. Reset clears validity so uninitialized targets cannot be used.

branch_predictor.v combines direction and target lookup to choose the next PC. Its query interface receives a known branch PC and a valid signal. Its update interface receives the resolved PC, outcome, target, and update-valid signal.

branch_recovery.v compares the saved next PC with the actual next PC. A mismatch on a valid resolution asserts flush and supplies redirect_pc. The consumer should act on the redirect only when recovery is valid.

Timing and reset behavior

Reads are combinational and updates happen on a rising clock edge. Active-high synchronous reset has priority over updates. The harness holds the original prediction until resolution, even if the current table output changes. It allows one outstanding branch and does not execute a complete instruction pipeline.

How we implement and run it

Build the simplest working path first

Start with always not taken, a small branch trace, and a recovery counter. This verifies trace input, PC + 4 selection, and flush behavior before adding state. Then add the 1-bit table and extend it to a 2-bit saturating counter using the same query and update interface.

Add target lookup and check recovery

Connect the tagged target buffer and save prediction metadata at issue. At N+2, compare the actual next address with the saved choice, then train the tables. Test missing targets, collisions, changed targets, and disabled updates before collecting performance results.

Use identical traces for every strategy

The generator creates always-taken and not-taken patterns, repeated and nested loops, alternating outcomes, fixed-seed biased random branches, history and target collisions, changing targets, and operand edge cases. Each record includes the branch PC, operands, target, outcome, and count of preceding nonbranch instructions.

The Python model independently predicts each branch and checks every RTL field. Each workload starts cold. The target-change trace is an adversarial interface test; a changing target at one PC is not typical of direct BEQ or BNE code.

Run from the project folder

```
make all
```

Runs tests, lint, trace generation, RTL comparison, synthesis, figures, and both reports. Required tools are Icarus Verilog, Verilator, Yosys, nextpnr-ice40, Python, and the Matplotlib environment documented in README.md.

```
make demo
```

Prints the repeated-loop comparison and a readable visit-by-visit walkthrough.

```
python3 scripts/demo.py --details
```

Adds readable counter states, target hits, and saved next addresses.

```
make sim
```

Runs the simulation path and regenerates reports without rerunning synthesis. An existing hardware report is retained. Open reports/results.html in a browser to inspect all workloads offline.

What the results show

The repeated-loop workload runs 32 invocations of nine taken outcomes followed by one not-taken exit. That gives 320 branches in 1,600 retired instructions. The comparison below uses cold tables, base CPI 1, and two extra cycles per recovery.

Strategy	Correct next PC	Recoveries	Estimated CPI
Always not taken	10.00%	288	1.36000
1-bit predictor	80.00%	64	1.08000
2-bit predictor	89.69%	33	1.04125

The baseline misses every taken outcome. The 1-bit predictor misses the first taken branch and the exit on every invocation. The 2-bit predictor misses the first training branch and each exit, giving 33 recoveries. It avoids 255 recoveries relative to the baseline on this workload.

$$CPI = 1 + \frac{recoveries \times 2}{1600}$$

This estimate counts recovery cost per retired instruction. General form: add base CPI to branch frequency multiplied by next-address miss rate and recovery penalty. The serialized harness cycle count must not be treated as measured CPU CPI.

Patterns that expose the limitations

On the alternating trace, always not taken gets 50% of next addresses correct; both dynamic predictors get 0% with our reset states. On target-buffer collisions, direction can be learned while next-address accuracy remains 0%, because the branches repeatedly evict each other's destinations.

Synthesis and timing estimates

Wrapped configuration	Logic cells	RAM blocks	Estimated MHz
Always not taken	592	2	93.92
1-bit predictor	961	4	77.37
2-bit predictor	1198	4	71.41
Serial fixture reference	337	0	324.36

Target: iCE40 HX8K CT256, seed 1, constraint 25 MHz. All configurations meet that constraint. Full designs include predictor, recovery, and registered serial transport. The fixture is a separate reference, not exact overhead to subtract. Frequencies are routed internal tool estimates, not physical-board measurements.

Verification and live demonstration

What the checks establish

The RTL matched the independent model on 7,248 branch predictions across 10 workloads and 3 strategies. The comparison checks state, direction, target hit, predicted address, actual address, and flush for every recorded branch.

The directed RTL testbench passes 90 checks, and 5 hand-calculated Python tests check known behavior. Directed cases cover all counter transitions and saturation, reset priority, disabled updates, query validity, table indexing, collisions, target replacement, saved predictions, invalid resolution, and address wraparound.

A serial wrapper makes FPGA evaluation possible with five external signals. Three transactions test cold prediction, learning, and the absence of repeated updates. Those transactions also pass for every mapped predictor in post-synthesis functional simulation. This check is not exhaustive equivalence or timing simulation.

A short presentation sequence

1. Explain the problem using the architecture diagram: choose the next address early and recover if it was wrong.

2. Run make demo. Compare 288, 64, and 33 recoveries on the same repeated loop.

3. Follow the first twelve visits. Point out the first exit and the following entry, where the 1-bit and 2-bit behavior differs.

4. Open the visual results page. Show the alternating trace and target-buffer collisions so the audience sees the limits.

5. Show the waveform and verification records. Finish with resource and timing estimates and explain what has been measured.

Where the evidence lives

Artifact	Location
Readable visual report	reports/results.html
Full technical report	reports/technical_report.md
Accuracy and recovery data	reports/accuracy.csv
Verification records	reports/verification.json and reports/unit_checks.json
Hardware and mapped checks	reports/synthesis/
Waveforms and signal list	reports/waveforms/ and docs/loop.gtkw

Requirements source: the supplied Project_list_MCA.pdf. Implementation details and figures: this project's RTL, tests, design notes, and measured CSV/JSON reports. No complete CPU, speculative rollback, multiple outstanding branches, or physical FPGA demonstration is included in this implementation.